Project Documentation

Introduction:

- Project title: product manager keep track of inventory.
- Team id:
- Team leader: Divyabharathi. P
  divya84008@gmail.com
- Team members:
- Monisha.S : monishasuresh0129@gmail.com
- Nadhiya. K
  nadhiyak981@gmail.com
- Madhumitha. B
  Madhubabu132007@gmail.com

1. **Abstract:**

- Product Inventory Management System is a backend application used to manage product details.
- It allows users to register, login and manage their products.
- Users can add, update, view and delete products easily.
- The system stores data securely in the database.

2. **Introduction:**
   - Inventory management helps businesses track their products and stock.
   - This system is developed to manage product details in a simple way.
   - It allows users to store product information such as name, price and quantity.
   - The system uses backend technologies to store and manage data.

3. Objectives:

- The main objectives of the project are:
- To develop a backend system for managing product inventory
- To implement secure user authentication
- To allow users to manage their own products

- To perform CRUD operations on products
- To store product information in a database
- To provide secure API access using authentication.

4. **Scope of the Project:**
   - The scope of this project is to develop a backend system that allows users to manage their product inventory.
   - The system focuses mainly on backend functionality. It provides API endpoints that can be used to perform different operations related to products.
   - Users can perform the following actions in the system:
   - Register a new account
   - Login to the system
   - Add new products
   - View their product list
   - Update product information
   - Delete products from the inventory
   - This project can be used as a learning project for students who want to understand backend development.
5. **Technologies Used:**

Technology	Purpose
Node.js	Used to run the backend server
Express.js	Used to create REST APIs
MongoDB	Used to store product data
Mongoose	Used to connect Node.js with MongoDB
JSON Web Token	Used for authentication
Bcryptjs	Used for password encryption

These technologies help create a secure and scalable backend system.

6. **System Architecture:**

The system follows a simple three-tier architecture.

1. **Client Layer:**

This layer represents the user who sends requests to the system.

2. **Backend Layer:**

The backend server processes user requests and performs operations.

3. **Database Layer:**

The database stores user information and product details.

System flow:

User → Backend Server → Database → Response to User

This architecture helps maintain a clean and organized system.

7.Backend Architecture:

- The backend system is built using Node.js and Express.js.
- The backend project follows a structured design pattern.

Controllers

- Controllers contain the main logic of the application. They handle user authentication and product operations.

Models

- Models define the structure of the database collections.

Routes

- Routes define API endpoints for different operations.
- Middleware
- Middleware is used to check authentication and validate requests.
- Configuration
- Configuration files store environment variables and database connection settings.

8.Database Architecture:

- The system uses MongoDB as the database.

- Two main collections are used in the database.
- Users Collection
- The Users collection stores information about registered users.

Fields include:

- ➢ User ID
- ➢ Name
- ➢ Email
- ➢ Password
- ➢ Created date
- ➢ Passwords are stored in encrypted form for security.

<u>Products Collection:</u>

The Products collection stores information about products.

Fields include:

- ✓ Product ID
- ✓ User ID
- ✓ Product name
- ✓ Price
- ✓ Quantity
- ✓ Description
- ✓ Created date
- ✓ Each product is linked to a specific user.

Relationship:

- ✓ One User → Many Products
- ✓ This means a single user can create multiple products.

<u>9</u>. <u>Core Features:</u>

- User Authentication
- The system provides secure user registration and login.
- Product Management
- Users can add new products to their inventory.

- Product Update
- Users can update product information like price or quantity.
- Product Deletion
- Users can delete products from their inventory.
- Product Search
- Users can search products by name.
- Secure API
- All APIs are protected using authentication middleware.

10. Roles and Responsibilities:

User

- ✓ The user is the main actor of the system.
- ✓ The user can perform the following tasks:
- ✓ Register a new account
- ✓ Login using credentials
- ✓ Add new products
- ✓ View product list
- ✓ Update product details
- ✓ Delete products
- ✓ Search products
- ✓ Users can access only their own product data.

11. User Flow:

- ❖ The working process of the system is as follows:
- ❖ The user sends a request to register or login.
- ❖ The backend verifies the user details.
- ❖ The system generates a JWT token.
- ❖ The user sends the token with API requests.
- ❖ Middleware verifies the token.
- ❖ Controllers perform product operations.
- ❖ MongoDB stores or retrieves product data.
- ❖ The server sends the response to the user.

12. Project Setup:

To start the project, the backend server must be configured.

Steps:

1. Create a backend project folder.
2. Initialize Node.js project.
3. Install required packages.
4. Create project folders.
5. Configure environment variables.
6. Connect MongoDB database.
7. Start the server.

13. API Testing:

- The APIs are tested using Thunder Client in Visual Studio Code.
- The following APIs are tested:
- Register User
- Allows a new user to create an account.
- Login User
- Allows a registered user to login and receive a JWT token.
- Create Product
- Allows a logged-in user to add a new product.
- Get Products
- Returns all products created by the user.
- Update Product
- Allows users to update product details.
- Delete Product
- Allows users to remove a product from the inventory.
- Testing APIs helps ensure that the system works correctly.

14. Advantages:

- The system provides several advantages:
- Easy inventory management
- Secure user authentication
- Fast backend API responses
- Organized data storage

- Easy product tracking
- Scalable backend architecture
- This system reduces manual work and improves efficiency.

15. Future Enhancements:

- The system can be improved in the future by adding more features.
- Possible improvements include:
- Product image upload
- Barcode scanning system
- Stock alert notifications
- Sales tracking system
- Mobile application integration
- Dashboard with reports
- These features can make the system more powerful and useful.

16. Conclusion:

The Product Inventory Management System provides a simple and secure way to manage product data. It allows users to register, login, and perform different operations on products.

Using modern backend technologies such as Node.js, Express.js, and MongoDB, the system provides fast and reliable API services.

This project helps users maintain product inventory efficiently and can also serve as a learning project for backend development.

Demo link:
https://drive.google.com/file/d/1lYxCtBMseLSjgjf9SSvLe2wimi8JyfLq/view?usp=drivesdk